DELOITTE CASE STUDY · CODE ANALYSIS & MIGRATION

Migrating Python to C++ with a local LLM pipeline

Analyze, categorize, convert and validate — without the code ever leaving the machine

Camille Monnier
Computer Engineering · Polytechnique Montréal · October 2026

Extract → Categorize → Convert → Validate

INTRODUCTION · THE CASE

From “syntax A to syntax B” to Python → C++17

What the case asks

- Analyze and categorize each legacy script
- Assess its complexity
- Convert it to the target syntax
- Validate the conversion by comparing outputs
- All of it with a local LLM or open-source NLP

My choice: A = Python, B = C++17

- **The pair I know best**, so I can judge every line the model writes
- **Secure by design:** local models only, the code never leaves the machine
- **Dynamic → static typing:** the hardest, most realistic part of a migration

INTRODUCTION · MY SOLUTION

A simple, hand-built pipeline

Plain Python and direct calls to local models through Ollama — no agent framework, no heavy dependencies. Each step is a small module that can be read and tested on its own.

DELIVERABLES

Documented C++ code

A .hpp / .cpp pair per script, plus the pybind11 bindings

Case: documented pipeline

Tests on the new code

The original Python tests re-run on the compiled C++

Case: comparison results

JSON trace files

Metadata, the model's assumptions, every prompt sent, test results

Case: output files

PIPELINE OVERVIEW

Four steps, two kinds of work



Deterministic where rules are reliable, LLM only where judgment is needed.

Deterministic LLM-powered

2.1 runs next to 2.2 for comparison; only the LLM categories feed the conversion.

STEP 1 · DETERMINISTIC, NO LLM

Read and profile the script

1 2.1 2.2 3 4

Parse with the AST

- File identity
- Structure (functions, main, CLI)
- Imports
- Key calls
- Inputs / outputs
- Business hints (names, strings)
- Metrics (branches, loops, nesting)
- Dynamic signals (eval, *args…)

Enrich types & libraries

Types: first source wins

1. Annotation in the code
2. MonkeyType (runtime traces)
3. Pyright (static inference)
4. Unknown → Any

Dependency mapping

```
numpy → Eigen · Medium
pandas → xtensor · High
math → <cmath> · Low
```

Score the risk per function

+2	per unknown type (parameter or return)
+10	if the file uses eval or getattr
+3	if the function takes *args or **kwargs

≥ 5 High | 2–4 Medium | < 2 Low

→ decides what Step 3 converts

Output: one profile JSON (identity, architecture, operations, dependencies, alerts, per-function details). Types and dependencies are what the next steps use most.

STEP 2.1 · DETERMINISTIC

Rule-based categorization

Technical category

Imports, method calls and the number of arithmetic operations trigger technical labels.

Data transformation | Computation / Algorithms | Utility

Business category

A keyword dictionary per domain, counted in names, strings and comments. The most cited domain wins.

Finance | Sales / Marketing | HR | Supply chain

Complexity score

Element	Points
Branches & loops	1 + ifs + loops
Nesting	+3 per level beyond 2
Size	+1 per 50 lines
Dependencies	+2 per import

≥ 30 Complex | **10–29 Moderate** | **< 10 Simple**

No description here: rules can't write a meaningful summary, so that is left to the LLM.

STEP 2.2 · LLM

LLM categorization, forced into JSON

01

Build the prompt

System: “you are an expert script analyst”, a rule for each field (description, technical, business, complexity) and one description example

User: the script + hints from Step 1

02

Constrain the output

- Ollama forces the JSON schema
- Qwen3.5-9B, fully on the GPU
- 16,384-token context
- Temperature 0, repeatable

03

Validate with Pydantic

The answer is parsed into typed models; categories must come from fixed lists.

Wrong shape → an error is raised, never silently accepted.

Independent from 2.1: the LLM gets raw facts from Step 1, never the rule-based answers — so the comparison stays fair.

STEP 3 · RULES FRAME IT, THE LLM WRITES THE BODIES

Convert function by function to C++17

a · Filter by risk

Low → convert

Medium → convert, review suggested

High → not converted, flagged

b · Impose signature

Types from convertion.json + how each parameter is used

```
pgcd(a, b)
int64_t pgcd(int64_t a, int64_t b)
```

c · Order the calls

Callees before callers, so every dependency's signature is known.

main comes last, renamed main_py.

d · Build the prompt

- Rulebook (system prompt)
- Function card + Python code
- Imposed C++ signature
- Signatures of its callees
- 2 examples of the theme
- Output: code + assumptions

e · Write the body

qwen3.6:35b-a3b writes only the function body, at temperature 0.

It also lists the assumptions it made.

f · Quick checks

- Signature unchanged
- No forbidden patterns
- Compiles (g++ -fsyntax-only)

g · Repair

Compiler errors go back into the same conversation.

2 attempts, then flagged — and its callers are skipped.

h · Save

.hpp / .cpp + pybind11 bindings

JSON: function cards, prompts, assumptions

STEP 4 · DETERMINISTIC

Validate against the original tests

1

Compile

The generated C++ is built into an importable Python module with pybind11.

2

Re-run the tests

The original tests run on the C++ functions. Unconverted ones stay in Python, so the tests still run.

3

Compare outputs

If main was converted, Python and C++ run side by side; console output is compared line by line.

Verdict: validated | differences found | module failed to compile

EXTRACTION · RULES VS LLM

Both methods on algorithms.py

Question	Rules (2.1)	LLM (2.2)
Business	Generic technical tool (default)	Generic technical tool · confidence 0.95
Technical	Utility + Computation / Algorithms	Computation / Algorithms, with justification
Complexity	Moderate · score 13, fully itemized	Simple · short, independent functions
Description	Not available	Math properties + palindrome check, printed to the console
Runtime	0.05 s	8 s

Same labels, better reasons

The LLM justifies its labels; the rules' business label is only the default.

Rules win on complexity

Every point is traceable, but the formula itself still needs tuning.

Description: LLM only

Rules can't summarize code — and the LLM is 160× slower.

CONVERSION RESULTS · A CASE THAT WORKS

algorithms.py: converted and validated

6/6

functions converted to C++

5/5

original tests passed on the C++

6/6

console lines identical to Python

Verdict: validated

ONE FUNCTION CARD

`fibonacci`

`status: converted · attempts: 1`

Assumptions written by the model

- The result fits in an int64_t (overflow possible for n ≥ 93)
- The input n follows Python semantics (non-negative integer)

Each function ships with its assumptions, so a reviewer knows exactly what to check.

CONVERSION RESULTS · A CASE THAT STOPS

A script built on the ast library

I fed the pipeline a piece of my own code: **compter_boucles**, a function that takes an **ast.Module** as input.

ast has no C++ equivalent in my mapping table, so no signature can be generated.

A controlled failure

Skipped and flagged, no LLM call wasted, and its callers are skipped too. Never silently wrong code.

Fix: map the type, or keep the function in Python — the hybrid module still runs the tests.

FUNCTION CARD


```
{
  "function": "compter_boucles",
  "status": "skipped",
  "reason": "type not convertible: ast.Module",
  "attempts": 0
}
```

CHALLENGES

Three challenges and how I addressed them

LANGUAGE **Dynamic Python → static C++**	A type mapping table, a support header that keeps Python semantics (// and % on negatives, str(), round()), and a flag on every risky change.
TECHNICAL CONSTRAINT **A local LLM means a small context window**	Split the script into functions and rewrite one at a time; a simple pipeline instead of a heavy multi-agent harness.
MODEL BEHAVIOR **The model doesn't always do what it's asked**	A detailed rulebook with explicit limits, examples from the same theme, an imposed signature — then checks and a repair loop.

CONCLUSION

What works today, and what comes next

TODAY

A working pipeline for single-file scripts: reliable on simple cases, with every decision traced.

Limit: larger, more complex code needs more context, and therefore more powerful hardware.

NEXT STEPS

1. Scale to multi-file projects and whole directories
2. Run on a stronger machine for a larger context
3. Use the LLM as a fallback for unknown types
4. Refine the complexity score

Thank you — questions?